

A Dual TF-IDF Linear SVM for Multi-Class News Article Classification

Francesco Borlandi and Luca Bosso

Politecnico di Torino

Student id: 310966 and 362957

Abstract—This report presents a linear approach for multi-class news article classification. The proposed solution combines dual TF-IDF representations, metadata features, and a calibrated Linear Support Vector Machine. Several targeted optimizations are applied to improve performance while preserving efficiency and interpretability. The final model achieves a macro-F1 score of 0.737 on the evaluation set.

I. PROBLEM OVERVIEW

The task addressed in this project is the automatic classification of news articles into multiple semantic categories. Each article is described by textual content (title and article body), categorical metadata (source), a timestamp, and a numerical attribute called page rank. The official evaluation metric for the assignment is macro-F1.

Figure 1 shows the distribution of class labels in the training set. The dataset exhibits moderate class imbalance, with some categories being significantly less frequent than others. This characteristic motivates the adoption of class-balanced learning strategies during model training.

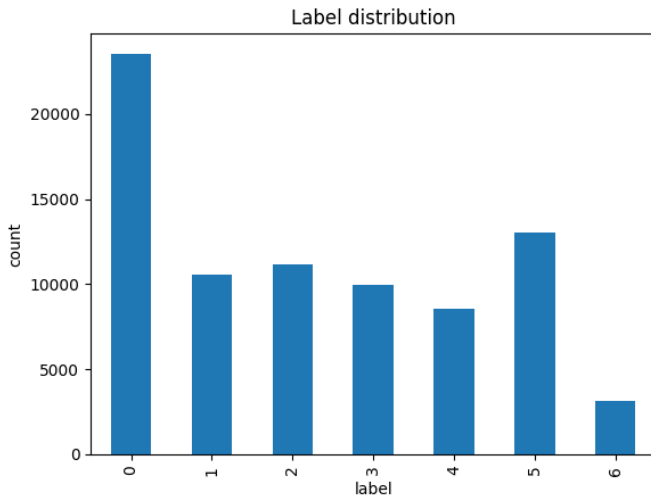


Fig. 1. Distribution of class labels in the training set.

Figure 2 reports the distribution of labels across news sources. A strong dependency between certain sources and specific labels can be observed, indicating that editorial style and publisher-specific patterns provide useful discriminative information for the classification task.

Figure 3 reports the distribution of document length across classes. The plot highlights that text length is not uniformly

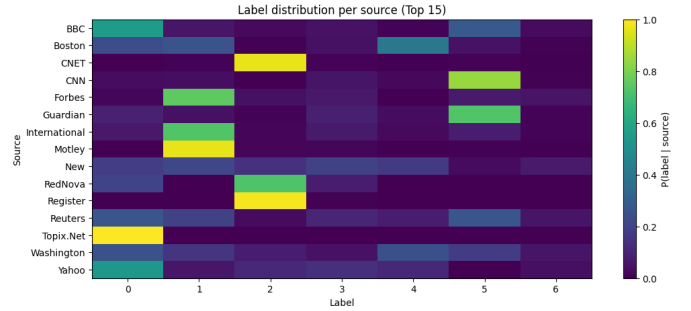


Fig. 2. Distribution of labels across news sources.

distributed among labels, with some categories being systematically associated with longer articles. This observation suggests that document length carries discriminative information and motivates its inclusion as an auxiliary feature in the classification pipeline.

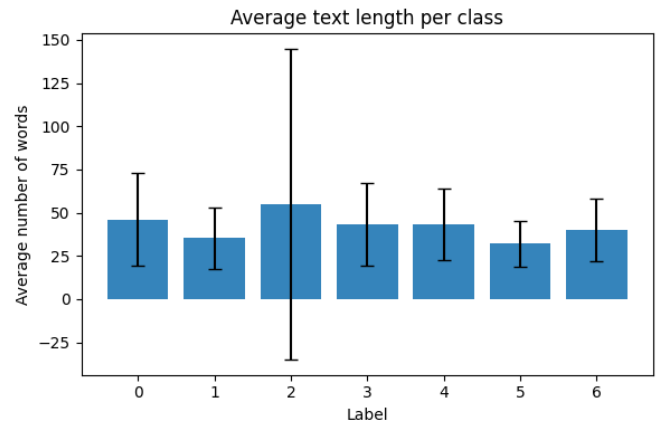


Fig. 3. Distribution of text length per class.

The dataset also includes a timestamp attribute. However, approximately 34.7% of the samples contain invalid timestamp values. In preliminary experiments, this feature was converted into a binary indicator encoding the presence of a valid timestamp. Since this transformation did not lead to measurable improvements, the timestamp was excluded from the final model.

Figure 4 shows the distribution of page rank values across classes. The plot reveals that all classes except label 2 are

almost exclusively associated with a page rank value of 5. In contrast, label 2 exhibits a broader distribution, spanning multiple page rank values. As a result, the page rank feature is mainly informative for distinguishing label 2 from the remaining categories, rather than providing global discriminative power.

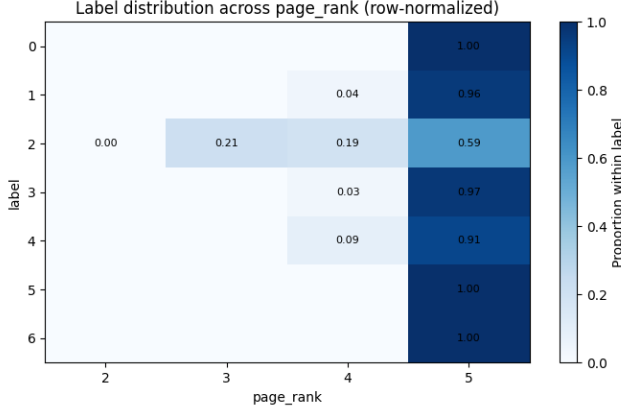


Fig. 4. Distribution of page rank across classes.

II. PROPOSED APPROACH

A. Preprocessing

The preprocessing stage is designed as a modular and reproducible feature pipeline that maps raw inputs to a structured set of feature blocks. Each sample is transformed into the concatenation of four distinct components: (i) a TF-IDF representation based on unigrams and bigrams, (ii) a weighted TF-IDF representation based on pure trigrams, (iii) one-hot encoded source metadata, and (iv) numerical features. This explicit feature contract guarantees consistency between training and evaluation phases and enables fine-grained control over the contribution of each block.

Textual fields (title and article body) are concatenated into a single text input. Missing values are replaced with empty strings to ensure robust tokenization and to prevent the propagation of missing values through the vectorization stage. Tokenization is restricted to alphabetic tokens only, reducing noise introduced by formatting artifacts, punctuation, or malformed tokens. English stopwords are removed, and terms appearing in more than a fixed fraction of documents are discarded to limit the impact of uninformative high-frequency tokens.

The main textual representation is based on TF-IDF features extracted from unigrams and bigrams. This block provides a robust and expressive baseline representation that captures local lexical patterns while remaining computationally efficient. A strict feature budget is enforced through a maximum number of features and minimum document frequency constraints, ensuring scalability and stability.

In addition, a secondary TF-IDF representation is built using pure trigrams. Higher-order n-grams are able to capture richer local context but are also substantially noisier

and sparser. Figure 5 shows a boxplot of document length across labels, highlighting significant differences in text length distributions between classes. Some categories are associated with systematically longer documents, where richer local context is available and higher-order n-grams can convey additional semantic information. This observation motivates the inclusion of trigrams as an auxiliary representation, rather than as a primary feature block. To mitigate sparsity and

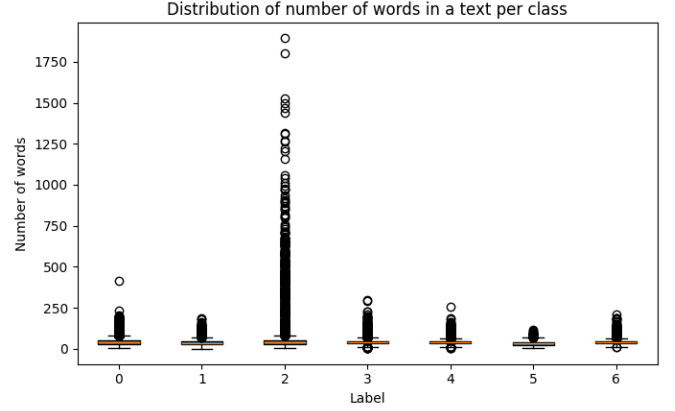


Fig. 5. Boxplot of document length (number of words) for each class.

noise, the trigram block is aggressively trimmed using higher minimum document frequency thresholds and a strict feature cap. Furthermore, its contribution is explicitly down-weighted by a scalar factor [1]. This weighting mechanism can be interpreted as computational budgeting, allowing informative trigrams to contribute selectively without overwhelming the main representation or increasing model variance [2].

Categorical metadata corresponding to the article source are encoded using one-hot encoding with explicit handling of unseen categories at inference time. Finally, two numerical features are included: page rank and document length, measured as the number of tokens. The inclusion of document length is directly motivated by the non-uniform distributions observed in Figure 5, while page rank is retained as a complementary signal for specific categories.

All feature blocks are combined using a column-wise transformer, producing a single sparse feature matrix that is passed to the classifier. This design balances expressiveness, robustness, and computational efficiency, while maintaining full transparency over the feature construction process.

B. Model selection

The final classifier selected for this task is a linear Support Vector Machine (SVM), implemented through the `LinearSVC` algorithm. This choice is motivated by both the characteristics of the feature space and the empirical behavior observed in preliminary model comparisons.

The TF-IDF representations employed in this work induce a high-dimensional and extremely sparse feature space. Linear SVMs are particularly well suited for this setting, as they can efficiently handle large sparse matrices while providing strong

generalization properties through margin maximization [3]. In contrast, tree-based models such as Decision Trees tend to struggle in high-dimensional sparse regimes, where decision splits become unstable and prone to overfitting, especially when many features are rarely active.

To support this choice, a lightweight empirical comparison was conducted between a linear SVM and a Decision Tree classifier on a stratified validation split, using the same pre-processing pipeline. The results are summarized in Table I. While the Decision Tree achieves a macro-F1 score of only 0.531, the linear SVM reaches a substantially higher macro-F1 of 0.721, confirming its superior suitability for sparse textual representations. Although the SVM incurs a moderate increase in computational time, the performance gain largely outweighs this cost in the context of the assignment.

TABLE I
COMPARISON BETWEEN CLASSIFIERS ON A STRATIFIED VALIDATION SPLIT.

Model	Macro-F1	Time (s)
Decision Tree	0.531	5.15
Linear SVM	0.719	11.26

An additional advantage of linear SVMs lies in their linear decision function, which provides direct access to class-wise decision scores. This property is essential for the post-hoc calibration step adopted in the proposed approach, where class-specific scaling factors and bias terms are applied to refine the final predictions without retraining the model. Such score-level manipulation is not naturally supported by tree-based classifiers.

The regularization parameter C controls the trade-off between margin maximization and training error. Class-balanced weights are enabled during training to mitigate the effect of class imbalance and ensure that minority classes contribute proportionally to the optimization objective.

Finally, the classifier is integrated into a unified pipeline together with the preprocessing stage. This design guarantees consistency between training and evaluation phases and allows the entire workflow to be reproduced in a single fit-predict procedure. After training on the full development set, predictions on the evaluation set are obtained by applying the calibrated decision function, where class-wise scaling and bias terms are used to optimize macro-F1 performance.

C. Hyperparameter tuning

Hyperparameter tuning was conducted through a structured and computationally efficient procedure. Rather than performing an exhaustive search, the exploration was intentionally restricted to a compact and carefully selected grid, guided by preliminary experiments and exploratory analyses.

In the first tuning stage, a joint grid search was performed over the regularization parameter C of the linear SVM and the scalar weight assigned to the auxiliary trigram TF-IDF block. This joint optimization was designed to balance model regularization with the contribution of higher-order n-grams,

which are known to provide richer contextual information but are also prone to sparsity and noise.

Figure 6 reports a heatmap of macro-F1 scores obtained on a stratified validation split for different combinations of C and trigram weight. The plot reveals a well-defined region of optimal performance, with a clear maximum attained for $C = 0.1$ and a trigram weight of $W = 0.5$. This configuration provides the best trade-off between regularization strength and auxiliary n-gram contribution, while maintaining stable performance across neighboring parameter values. Once the

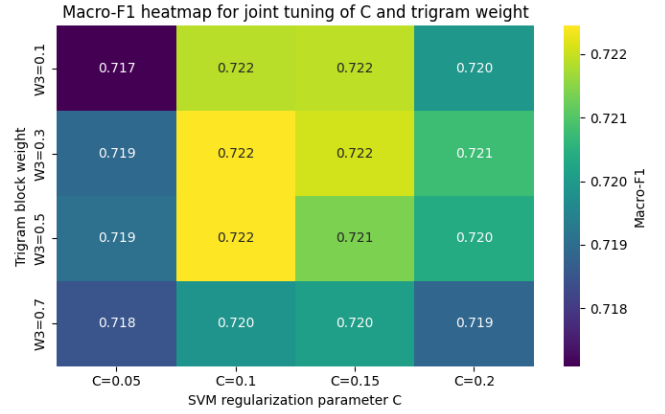


Fig. 6. Macro F1 Heatmap for joint tuning of C and trigram weight

optimal values of C and the trigram weight were fixed, a second tuning stage focused on post-hoc calibration of the classifier decision scores. In this phase, class-specific scaling factors (α) and bias terms were estimated on validation splits. This calibration step directly adjusts the decision function outputs without requiring retraining of the model and proved effective in further improving macro-F1 by refining class decision boundaries.

The final model configuration is obtained by combining the grid-selected hyperparameters with the calibrated α and bias values. This configuration is then used to train the classifier on the full development set and to generate predictions on the evaluation data.

III. RESULTS

This section presents the experimental results obtained with the proposed classification pipeline. All model comparisons are conducted using macro-F1, in accordance with the official evaluation metric of the assignment.

Table II reports an ablation study performed on two stratified holdout splits of the development set. The baseline model, based on TF-IDF unigrams and bigrams combined with metadata features, achieves a macro-F1 score of 0.722. Introducing pure trigram features without weighting leads to a performance degradation, confirming that higher-order n-grams are inherently sparse and noisy when used without additional constraints.

A clear improvement is observed when trigram features are down-weighted and treated as an auxiliary representation. This

TABLE II
ABLATION STUDY ON TWO STRATIFIED HOLDOUT SPLITS.

Model variant	Macro-F1
Baseline TF-IDF (1,2) + metadata	0.719
+ Trigram features (3,3)	0.716
+ Weighted trigrams	0.722
+ Calibration (alpha + bias)	0.725

configuration yields a macro-F1 score of 0.729, indicating that selectively incorporating higher-order context can provide complementary information without overwhelming the main feature space. A large performance gain is obtained through post-hoc calibration of the classifier scores using class-specific scaling (alpha) and bias terms, resulting in a macro-F1 score of 0.732 on the development set.

Figure 7 shows the normalized confusion matrix computed on a validation split. Most classes exhibit strong diagonal dominance, indicating balanced classification behavior. The most challenging categories are labels 3 and 5, which are frequently confused with label 0. In particular, label 5 is often misclassified as label 0, a behavior that can be attributed to semantic overlap between the corresponding categories rather than to systematic model bias.

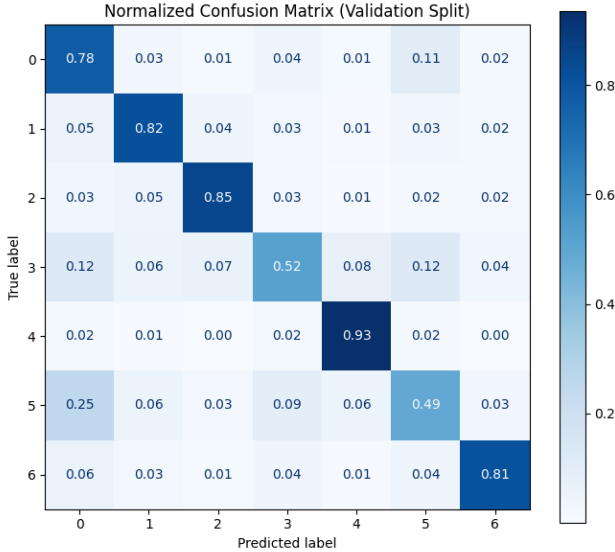


Fig. 7. Normalized confusion matrix on a validation split.

The final model is trained on the full development set and evaluated on the held-out evaluation set. It achieves a macro-F1 score of **0.74**, demonstrating strong generalization performance. End-to-end prediction on the evaluation set requires 20.37 seconds, confirming that the proposed approach remains computationally efficient despite the use of multiple feature blocks and post-processing calibration.

IV. DISCUSSION

The results show that higher-order n-grams can provide useful contextual information but require careful weighting to

avoid introducing noise. Metadata features, particularly source and document length, complement textual representations and improve robustness.

Post-hoc calibration proves to be an effective and computationally inexpensive strategy for improving macro-F1 in multi-class settings. Overall, the results demonstrate that a well-engineered linear model remains competitive for large-scale text classification tasks.

REFERENCES

- [1] G. Salton and C. Buckley, "Term-weighting approaches in automatic text retrieval," *Information Processing & Management*, vol. 24, no. 5, pp. 513–523, 1988.
- [2] D. Jurafsky and J. H. Martin, *Speech and Language Processing*. Prentice Hall, 2009.
- [3] C. Cortes and V. Vapnik, "Support-vector networks," *Machine Learning*, vol. 20, no. 3, pp. 273–297, 1995.